Optical Character Recognition

Author A (Student ID: XXXXXXX)
Author B (Student ID: XXXXXXX)
Author C (Student ID: XXXXXXX)
Author D (Student ID: XXXXXXX)

42.515 Machine Learning and Analytics
Singapore University of Technology and Design
Spring 2026

April 12, 2026

Contents

1 Motivation **3**

2 Context **3**
2.1 Problem setting and pipeline role 3
2.2 Historical progression 3
2.3 Precedent topics 4
2.4 Successor topics 4
2.5 Tangential topics 4

3 Theory **4**
3.1 Convolutional neural networks 4
3.2 Recurrent neural networks 4
3.3 Text detection 5
3.4 Text recognition: from image columns to character posteriors 6
3.5 Connectionist Temporal Classification (CTC) 6
3.6 Decoding and language-informed search 7

4 Evaluation Metrics **7**
4.1 Text detection (Mask R-CNN) 7
4.2 Text recognition (CRNN + CTC) 8
4.3 End-to-end pipeline metrics 8

5 References **9**

1 Motivation

Whiteboards are ubiquitous in collaborative environments: meeting rooms, lecture halls, and design studios. The content produced there—handwritten notes, diagrams with annotations, equations, and sketches—often encodes decisions, rationales, and tacit knowledge that is costly to reconstruct later. That content is also inherently *ephemeral*: once the board is erased, the trace disappears unless it has been captured in a durable, machine-readable form.

Photographing a board is easy, but a photo is only a passive bitmap. It cannot be searched by keyword, quoted precisely in reports, diffed across versions, or linked into task trackers and knowledge bases. The gap between "we have an image" and "we have usable text" is exactly what Optical Character Recognition (OCR) is meant to close.

This report adopts a standard two-stage deep learning pipeline that mirrors how humans read a cluttered board: first find candidate text regions, then read each region as a sequence of characters.

1. **Text detection (Mask R-CNN):** predict instance-level regions (bounding boxes and optional masks) so each text line or word can be cropped for recognition.

2. **Text recognition (CRNN + CTC):** map each crop to a character string without hand-designed character segmentation, using a convolutional front end and a recurrent encoder trained with Connectionist Temporal Classification (CTC).

Decomposing the problem improves interpretability (we can diagnose detection versus recognition failures), reuse (the same recognizer can run on other detectors), and data efficiency (detection and recognition can be pre-trained on different corpora). The motivation is therefore not only accuracy in isolation, but a maintainable pipeline whose errors can be measured and improved stage by stage.

2 Context

2.1 Problem setting and pipeline role

OCR sits at the boundary of computer vision and sequence modeling. The input is an image (or a region thereof); the output is structured text, often with an implicit reading order. In document and scene OCR, the dominant paradigm is still *detect-then-recognize*: a detector proposes regions, and a recognizer consumes fixed-height crops rescaled to the recognizer's input geometry [1], [2]. That decomposition remains relevant even as end-to-end document models emerge [3], because modular pipelines are easier to debug, swap, and deploy under compute constraints.

For whiteboard-like imagery, detection must tolerate free-form layout and overlapping ink, while recognition must tolerate stroke variability and low contrast. Mask R-CNN [4] is a natural detection choice when instance masks help separate touching or densely packed lines; CRNN [2] is a mature recognition backbone when labels are strings without per-character time alignments.

2.2 Historical progression

At a high level [2], [5], [6], [7]:

1. Template and rule-based systems with heavy preprocessing.

2. Statistical models with engineered features and explicit or implicit segmentation.

3. CNN–RNN hybrids with CTC or attention for alignment-free or soft-aligned training.

4. Transformers and large-scale pretraining for recognition and document understanding [3], [8].

2.3 Precedent topics

OCR inherits ideas from classical image processing and pattern recognition: binarization, connected components, skew correction, and template- or feature-based classifiers [5]. Industrial engines such as Tesseract historically combined carefully engineered pipelines with adaptive thresholding and language models at decode time [6]. Statistical sequence models (for example Hidden Markov Models) already treated recognition as alignment between observations and discrete labels, foreshadowing modern sequence losses.

Deep learning shifted the emphasis from hand-crafted features to learned representations. CNNs supply translation-tolerant visual features; recurrent or attention-based heads model the left-to-right structure of text in a crop [2], [9]. Surveys of scene text detection and recognition summarize the explosion of architectures and benchmarks in the deep learning era [1].

2.4 Successor topics

Once text is extracted, it feeds downstream systems: information retrieval over meeting archives, automatic minutes, accessibility (screen readers), and retrieval-augmented generation where scanned or photographed boards become part of a corpus. Layout-aware models additionally use spatial relations between blocks [3]. Stronger language models at decode time further blur the line between "vision-only" recognition and NLP-style correction.

2.5 Tangential topics

- **Handwritten text recognition (HTR):** larger stroke-level variability than printed OCR; datasets such as IAM support line-level supervision [10].
- **Scene text:** perspective, blur, and background clutter; benchmark analyses highlight how evaluation protocol affects reported gains [7], [11].
- **Language-aware decoding:** lexicons, n-gram or neural language models, and beam search bias hypotheses toward fluent text [2].

3 Theory

3.1 Convolutional neural networks

A convolutional neural network (CNN) stacks convolution layers that apply learned filters across spatial locations. Shared weights enforce translation equivariance: the same feature responds to a pattern regardless of small shifts in the input. Nonlinearities and pooling (or strided convolutions) build a hierarchy from edges and strokes to parts and whole glyphs. For OCR, CNNs therefore reduce sensitivity to slight misalignment of a crop while still encoding local stroke structure.

In both Mask R-CNN and CRNN, a CNN acts as a *backbone* or *visual encoder*: it maps the input image to feature maps that downstream heads interpret as either object proposals (detection) or a sequence of column-wise descriptors (recognition).

3.2 Recurrent neural networks

A recurrent neural network (RNN) maintains a hidden state updated at each time step, allowing the model to carry context along a sequence. For text recognition, the sequence index often corresponds to horizontal position in a feature map after the CNN. Unidirectional RNNs use past context only; bidirectional RNNs (for example bidirectional LSTMs) combine forward and backward passes so each position conditions on both sides of a character. That is valuable for

ambiguous characters (`0` vs `O`, `l` vs `I`) where local appearance is insufficient. Gated units (LSTM, GRU) mitigate vanishing gradients over long spans compared with plain tanh RNNs.

3.3 Text detection

Text detection answers *where* to read. Unlike full-image classification, the model must output a variable number of regions, each with a confidence score and geometry. General object detectors adapt well to "text" as a class: they learn discriminative boundary cues (ink contrast, stroke alignment) and suppress background (markers, wall texture, diagrams). Surveys of scene text organize methods by representation—regression of oriented boxes, segmentation of text regions, or hybrid anchor/refinement pipelines—and emphasize that failure modes differ by layout, script, and resolution [1].

Supervision granularity and targets. Labels may be words, lines, or arbitrary quadrilaterals. Line-level boxes match how many sequence recognizers consume crops (one baseline per crop); word-level boxes increase instance count and boundary ambiguity at spaces. Axis-aligned rectangles are simple to annotate and fast to evaluate with IoU, but they waste area on slanted or curved lines; rotated or quadrilateral targets tighten the envelope at the cost of more complex regression and NMS. For whiteboards, free-form placement and long horizontal lines favor predictors that tolerate large aspect ratios and occasional overlap between neighboring instances.

Multi-scale features. Text strokes vary from sub-pixel thickness in distant photos to dominant structure in close-ups. Modern detectors therefore build a *feature pyramid* from the backbone (often with lateral connections and top-down pathways) so that small text is still represented at high spatial resolution while large regions draw context from semantically strong low-resolution maps. Proposal networks and heads consume pyramid levels jointly, which stabilizes recall for both fine print and headings on the same board.

Proposal-based two-stage detection. Region-based CNNs follow a coarse-to-fine pattern: a backbone builds a multi-scale feature pyramid; a region proposal network (RPN) classifies anchor boxes as foreground/background and regresses box refinements; a second stage pools features inside each proposal and predicts class labels and refined boxes. Non-maximum suppression (NMS) then removes redundant high-overlap predictions. For elongated text, anchor banks are usually skewed toward wide, short boxes on each pyramid level so that the RPN does not systematically miss lines that would require extreme aspect ratios from square-ish defaults.

Mask R-CNN. Mask R-CNN extends Faster R-CNN with a parallel *mask head* that predicts a binary mask inside each region of interest, using RoIAlign to avoid quantization artifacts when sampling features from the feature map [4]. RoIAlign uses bilinear interpolation on continuous coordinates so that small box shifts do not snap to the wrong feature bins—a practical issue when box width is only a few pixels per character. The multi-task loss combines classification, bounding-box regression, and per-pixel mask loss (typically binary cross-entropy). For text, masks can follow irregular ink more tightly than axis-aligned rectangles alone—useful for curved baselines, tightly packed lines, or partial occlusion on a whiteboard.

Related detector families. Single-stage detectors such as YOLO [12] regress boxes and class scores in one pass and are often faster at similar input resolutions, though very small instances can demand careful tuning. Segmentation-based methods predict a text probability map and decode boxes or polygons afterward; differentiable binarization (DB) trains tight boundaries and is widely used when low latency matters [13]. For whiteboard ink that touches across lines, this report still favors Mask R-CNN when instance masks are available to separate instances before recognition.

Training objective. With classes $\mathcal{C}$ (including "text"), bounding boxes B, and mask targets M, training minimizes a weighted sum of losses:

$$\mathcal{L}_{\text{Mask R-CNN}} = \mathcal{L}_{\text{RPN}} + \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}.$$

Here $\mathcal{L}_{\text{box}}$ encourages tight localization (smooth L1 or related regression loss on box parameters), while $\mathcal{L}_{\text{mask}}$ encourages pixel-accurate coverage of text strokes inside each positive detection.

Post-processing. After inference, detections are filtered by score threshold, NMS merges overlapping boxes, and crops (optionally masked or padded to a fixed aspect ratio) are passed to the recognizer. Score calibration matters for downstream matching: overconfident duplicate boxes waste recognizer compute and can duplicate text in the transcript unless suppressed by IoU-aware NMS or mask-aware merging. Weak detection—missed lines, merged regions, or shifted boxes—directly causes recognition errors even when the recognizer is strong, because CTC-based recognizers assume a single dominant line per crop and degrade when two strings overlap spatially.

3.4 Text recognition: from image columns to character posteriors

Let a detector crop (or masked patch) be X. Recognition outputs a string y over an alphabet $\mathcal{V}$. The CRNN architecture [2] chains three ideas: (i) extract a deep feature map with a CNN, (ii) treat horizontal position as discrete time, and (iii) model inter-column dependence with a recurrent encoder before emitting a distribution over $\mathcal{V} \cup \{\varnothing\}$ at each time step.

CNN feature map. The CNN maps X to a tensor with reduced height and width. Height is often collapsed to one or few rows so that each spatial column aggregates vertical evidence across the text line. Width T then indexes positions along the line; longer lines yield larger T after fixed-scale resizing, which is how variable-length text is represented without resizing the string.

Column-wise sequence. Let $H = f_\theta(X) = (h_1, \ldots, h_T)$ be the sequence of column descriptors read left to right. Each $h_t \in \mathbb{R}^d$ summarizes local appearance at that horizontal location.

BiLSTM encoding. A bidirectional RNN computes forward and backward hidden states and concatenates them:

$$\overrightarrow{s_t} = \mathrm{RNN}_f(h_t, \overrightarrow{s_{t-1}}), \qquad \overleftarrow{s_t} = \mathrm{RNN}_b(h_t, \overleftarrow{s_{t+1}}), \qquad s_t = [\overrightarrow{s_t}; \overleftarrow{s_t}].$$

The emitted vector s_t encodes both local visual evidence and contextual neighbors, reducing confusion between visually similar characters.

Per-step posteriors. A linear layer with softmax yields frame-level probabilities

$$P(\pi_t = k \mid X) = \mathrm{softmax}(W s_t + b)_k, \qquad k \in \mathcal{V} \cup \{\varnothing\},$$

where $\varnothing$ is the CTC blank. These T distributions are the only per-frame supervision needed: no boundary labels for where each character starts or ends.

3.5 Connectionist Temporal Classification (CTC)

CTC [14] trains sequence models when the alignment between time steps $t = 1, \ldots, T$ and label indices is unknown. Let $y = (y_1, \ldots, y_{|y|})$ be the target string. A *path* $\pi = (\pi_1, \ldots, \pi_T)$ with $\pi_t \in \mathcal{V} \cup \{\varnothing\}$ is a frame-wise label sequence. Define the collapse map $\mathcal{B}$ that (1) removes blanks and (2) merges consecutive identical symbols. For duplicate characters in y (for example "hello"), valid paths must insert a blank between repeated letters so that collapse does not merge them incorrectly; the dynamic-programming construction of allowed transitions encodes this constraint.

Path likelihood. Assuming conditional independence across time given the recurrent state (the standard CTC graphical model),

$$P(\pi \mid X) = \prod_{t=1}^{T} P(\pi_t \mid X).$$

The probability of string y is the sum over all paths that collapse to y:

$$P(y \mid X) = \sum_{\pi \in \mathcal{B}^{-1}(y)} P(\pi \mid X), \qquad \mathcal{L}_{\text{CTC}} = -\log P(y \mid X).$$

The set $\mathcal{B}^{-1}(y)$ is enormous, but $P(y \mid X)$ is computed in $O(T\,|y|)$ by dynamic programming. Let y' denote the label sequence expanded with blanks following the CTC topology. Forward variables $\alpha_t(j)$ accumulate probability mass for prefixes of y' ending in state j at time t; backward variables $\beta_t(j)$ play the symmetric role. Transitions respect whether the next symbol is a blank or a character and whether repeated characters require a blank spacer. Implementations use log-space arithmetic for numerical stability.

Why CTC suits OCR. Characters vary in width; the same letter may span several columns after pooling. CTC does not force a one-to-one link between characters and fixed-width windows. Instead, the network may emit blanks until evidence supports the next character, spreading probability mass over many valid alignments during early training.

3.6 Decoding and language-informed search

Training optimizes $P(y \mid X)$, but inference requires choosing one string $\hat{y}$. Shi et al. [2] discuss decoding for image-based sequence recognition with CTC in both *lexicon-free* and *lexicon-constrained* regimes (their CRNN experiments on scene text use both).

Greedy (best-path) decoding. The simplest rule takes the most likely label at each frame, $\pi_t = \arg\max_k P(\pi_t{=}k \mid X)$, then applies the CTC collapse $\mathcal{B}$. This is fast but can be suboptimal: locally best symbols need not maximize $P(y \mid X)$ after collapse [2].

Beam search. Keeping the top B partial hypotheses by cumulative score explores alternatives that per-step greedy search discards. Shi et al. [2] use beam search when scoring candidates against a lexicon or when approximate search over a large word list is needed.

Lexicon and language priors. When a dictionary $\mathcal{L}$ is known (per-image word list or application vocabulary), decoding restricts $\mathcal{Y} \subseteq \mathcal{L}$ or re-ranks hypotheses using lexical overlap with the CTC output [2]. A convenient scoring form combines the CTC string likelihood with a language or lexicon model P_{LM} over the candidate string:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \Big[\log P_{\text{CTC}}(y \mid X) + \lambda \log P_{\text{LM}}(y) + \beta\,|y| \Big],$$

where $\mathcal{Y}$ is the beam or lexicon; λ weights the linguistic prior relative to the visual model; and β can penalize length [2]. In the open-vocabulary case, $\mathcal{Y}$ is unconstrained other than the beam; with a closed lexicon, $\mathcal{Y} = \mathcal{L}$.

4 Evaluation Metrics

Evaluation should reflect the pipeline: detection quality, recognition quality on isolated crops, and end-to-end behavior when both stages compose.

4.1 Text detection (Mask R-CNN)

Let each ground-truth instance have a bounding box (and optionally a mask polygon). Predictions are matched to ground truth using Intersection over Union (IoU) on boxes, or mask IoU if masks are evaluated:

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}.$$

A prediction is a true positive (TP) if it is matched to an unmatched ground-truth object with IoU $\geq \tau$ (common $\tau \in \{0.5, 0.75\}$) and the predicted class is correct. Unmatched predictions are false positives (FP); unmatched ground truths are false negatives (FN).

Precision, recall, and F1 at a fixed IoU:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Average precision (AP) and mean AP (mAP): For a given IoU threshold, sort predictions by confidence and plot precision versus recall as the score threshold varies; AP is the area under this curve. mAP averages AP across classes (for single-class "text", mAP reduces to AP for that class). COCO-style evaluation reports mAP@0.5 and mAP@0.5:0.95 (AP averaged over IoU from 0.5 to 0.95), which rewards both localization and ranking quality [4].

Mask-specific quality. When masks are used to separate touching lines, mean mask IoU on matched instances measures pixel-level tightness beyond axis-aligned box IoU.

4.2 Text recognition (CRNN + CTC)

Recognition metrics assume a protocol: either (i) **isolated crop evaluation**, where crops are generated from ground-truth boxes so detector errors do not affect the score, or (ii) **detector-provided crops**, which mixes recognition with crop quality.

Let S, D, and I denote substitutions, deletions, and insertions from the Levenshtein alignment between hypothesis and reference, and let N be the number of reference characters (for CER) or words (for WER).

Character Error Rate (CER):

$$\text{CER} = \frac{S + D + I}{N_{\text{chars}}}.$$

Word Error Rate (WER): same formula at word level with N_{words} reference words.

Sequence accuracy: exact match rate is the fraction of crops for which CER $= 0$ (equivalently hypothesis equals reference). **Normalized edit distance** is sometimes reported as $1 - \text{CER}$ for interpretability.

Calibration-style diagnostics (optional): average CTC entropy per frame, or frame-wise max-probability, can indicate overconfident but wrong predictions when CER is high.

4.3 End-to-end pipeline metrics

When detection and recognition run sequentially, errors compound: a missed detection is a deletion of an entire string; a duplicate detection duplicates content; a merged box concatenates two references into one hypothesis.

Matching-based protocol. A common approach is: (1) match predicted boxes to reference boxes with IoU $\geq \tau$ (Hungarian matching or greedy one-to-one); (2) for each matched pair, compare recognized text to the reference string for that box (CER/WER); (3) count unmatched predictions as insertions of spurious text and unmatched references as deletions of missed text. Aggregating yields a **micro-averaged** word or character error over the document, and **macro-averaged** per-line statistics.

Full-page metrics. For ordered reading, some benchmarks measure edit distance on the concatenation of text in reading order; this penalizes wrong ordering even when local boxes are correct.

Detection-aware recognition. Define *recognition rate given correct localization*: mean CER only on pairs with IoU above τ, to isolate the recognizer. Conversely, report *word recall* at the pipeline output: fraction of reference words that appear in any hypothesis after matching—this stresses detection.

Frequently used corpora include IAM for handwriting [10], ICDAR-style scene and document benchmarks, and synthetic pretraining sets analyzed in STR studies [7], [11].

Table 1: Metrics by pipeline stage (summary)

Stage	Primary metrics	What they reveal
Detection (Mask R-CNN)	AP / mAP, Precision/Recall/F_1 at IoU τ, mask IoU	Localization, duplicates, misses
Recognition (CRNN)	CER, WER, exact match (on GT or detector crops)	Reading quality independent of or coupled to crops
End-to-end	Matched-pair CER/WER, unmatched FP/FN counts, page-level edit distance	Compounding effects and reading order

5 References

[1] S. Long, X. He, and C. Yao, "Scene text detection and recognition: The deep learning era," *International Journal of Computer Vision*, vol. 129, no. 1, pp. 161–184, 2021.

[2] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017. DOI: 10.1109/TPAMI.2016.2646371

[3] G. Kim et al., "OCR-free document understanding transformer," in *European Conference on Computer Vision (ECCV)*, 2022, pp. 498–517.

[4] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969. DOI: 10.1109/ICCV.2017.322

[5] S. Mori, C. Y. Suen, and K. Yamamoto, "Historical review of OCR research and development," *Proceedings of the IEEE*, vol. 80, no. 7, pp. 1029–1058, 1992.

[6] R. Smith, "An overview of the tesseract ocr engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, vol. 2, 2007, pp. 629–633.

[7] J. Baek et al., "What is wrong with scene text recognition model comparisons? dataset and model analysis," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 4715–4723.

[8] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, pp. 13 094–13 102, 2023.

[9] M. Jaderberg, K. Simonyan, A. Vedaldi, and A. Zisserman, "Reading text in the wild with convolutional neural networks," *International Journal of Computer Vision*, vol. 116, no. 1, pp. 1–20, 2016.

[10] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002. DOI: 10.1007/s100320200071

[11] M. Li et al. "TRBA: A benchmarking platform for scene text recognition." arXiv: 2107.11065.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91

[13] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 11 474–11 481.

[14] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 369–376. DOI: 10.1145/1143844.1143891